Déploiement d'une application Next.js sur OVH Public Cloud

Ce guide détaille toutes les étapes pour déployer une application Next.js sur OVH Public Cloud de manière professionnelle et scalable.

1. Préparation du projet

Configuration Docker

Créez un fichier `Dockerfile` à la racine de votre projet:

```
FROM node:18-alpine AS base

# Dossier de travail
WORKDIR /app

# Installation des dépendances
FROM base AS deps
COPY package.json package-lock.json* ./
RUN npm ci

# Construction de l'application
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
# Génération du modèle Prisma et build de l'application
RUN npx prisma generate
RUN npm run build

# Image de production
FROM base AS runner
WORKDIR /app

ENV NODE_ENV production

# Création d'un utilisateur non-root
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs
USER nextjs

# Copie des fichiers nécessaires
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

# Exposition du port 3000
EXPOSE 3000

# Variable d'environnement pour écouter sur toutes les interfaces
```

```
ENV PORT 3000
ENV HOSTNAME "0.0.0.0"

# Commande de démarrage
CMD ["node", "server.js"]
```

Configuration Next.js

Modifiez votre fichier `next.config.js` ou `next.config.ts` pour activer le mode standalone:

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  output: "standalone", // Important pour Docker
  images: {
    // Configuration existante...
  },
};

export default nextConfig;
```

Fichier .dockerignore

Créez un fichier `.dockerignore` pour exclure les fichiers inutiles:

```
# Dépendances
node_modules
npm-debug.log

# Build
.next
out

# Système
.DS_Store
*.pem

# Debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Local env files
.env
.env.local
.env.development.local
.env.test.local
.env.production.local
.env*.local

# Contrôle de version
```

```
.git
.github
.gitlab

# Divers
README.md
*.md
LICENSE
.editorconfig
*.log
.vercel
```

2. Configuration d'une instance OVH Public Cloud

Création de l'instance

1. Connectez-vous à votre compte OVH et accédez au Public Cloud
2. Créez une instance avec les caractéristiques suivantes:
    - OS: Ubuntu 22.04 LTS
    - Type: B2-7 (2 vCores, 7 Go RAM) minimum
    - Stockage: SSD 20 Go minimum
    - Réseau: créez et associez une IP publique

Configuration initiale du serveur

Connectez-vous à votre instance via SSH:

```
ssh ubuntu@VOTRE_IP_PUBLIQUE
```

Mettez à jour le système et installez Docker:

```
# Mise à jour du système
sudo apt update && sudo apt upgrade -y

# Installation de Docker et Docker Compose
sudo apt install -y docker.io docker-compose

# Configuration des droits Docker
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ubuntu

# Reconnectez-vous au serveur pour appliquer les changements de groupe
exit
ssh ubuntu@VOTRE_IP_PUBLIQUE
```

Créez la structure de répertoires pour le déploiement:

```
mkdir -p ~/app/nginx/{conf.d,ssl,logs,certbot}
```

3. Fichiers de configuration pour le déploiement

Fichier docker-compose.yml (version complète)

Créez un fichier `docker-compose.yml` :

```yaml
version: "3.8"

services:
  nextjs:
    build:
      context: .
      dockerfile: Dockerfile
    restart: always
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      # Variables d'environnement de production
      - DATABASE_URL=${DATABASE_URL_PRODUCTION}
      - NEXTAUTH_SECRET=${NEXTAUTH_SECRET}
      - NEXTAUTH_URL=https://votre-domaine.com # À modifier avec votre domaine
    volumes:
      - ./uploads:/app/uploads
    depends_on:
      - proxy

  proxy:
    image: nginx:alpine
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d
      - ./nginx/ssl:/etc/nginx/ssl
      - ./nginx/logs:/var/log/nginx
    depends_on:
      - certbot

  certbot:
    image: certbot/certbot
    volumes:
      - ./nginx/ssl:/etc/letsencrypt
      - ./nginx/certbot:/var/www/certbot
    command: certonly --webroot --webroot-path=/var/www/certbot --email
admin@example.com --agree-tos --no-eff-email -d votre-domaine.com
```

Configuration Nginx (avec SSL)

Créez un fichier `nginx.conf` :

```
server {
    listen 80;
    server_name votre-domaine.com;

    # Redirection vers HTTPS
    location / {
        return 301 https://$host$request_uri;
    }

    # Pour le renouvellement de certificat Let's Encrypt
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}

server {
    listen 443 ssl http2;
    server_name votre-domaine.com;

    # Certificats SSL
    ssl_certificate /etc/nginx/ssl/live/votre-domaine.com/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/live/votre-domaine.com/privkey.pem;

    # Paramètres SSL optimisés
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES128-GCM-SHA256;
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    # HSTS (HTTP Strict Transport Security)
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;

    # Autres en-têtes de sécurité
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";

    # Proxy principal vers l'application Next.js
    location / {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
```

```
        # Timeouts plus longs pour les opérations qui durent longtemps
        proxy_read_timeout 60s;
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
    }

    # Mise en cache des assets statiques
    location /_next/static/ {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;

        # Cache pour les fichiers statiques
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
        access_log off;
    }

    # Mise en cache des images statiques
    location /static/ {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;

        # Cache pour les images et autres médias
        expires 30d;
        add_header Cache-Control "public, max-age=2592000";
        access_log off;
    }
}
```

Variables d'environnement de production

Créez un fichier `.env.production` :

```
# Variables de production
NODE_ENV=production

# Base de données
DATABASE_URL_PRODUCTION=postgres://user:password@host:port/database?sslmode=require

# NextAuth
NEXTAUTH_SECRET=votre_secret_ici
NEXTAUTH_URL=https://votre-domaine.com

# Variables publiques
NEXT_PUBLIC_API_URL=https://votre-domaine.com/api
```

Configuration initiale (sans SSL)

Pour démarrer sans avoir configuré le DNS, créez un fichier `docker-compose-initial.yml` :

```yaml
version: "3.8"

services:
  nextjs:
    build:
      context: .
      dockerfile: Dockerfile
    restart: always
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=${DATABASE_URL_PRODUCTION}
      - NEXTAUTH_SECRET=${NEXTAUTH_SECRET}
      - NEXTAUTH_URL=http://VOTRE_IP_PUBLIQUE:3000
    volumes:
      - ./uploads:/app/uploads

  # Proxy simplifié sans SSL pour la configuration initiale
  proxy:
    image: nginx:alpine
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d
      - ./nginx/logs:/var/log/nginx
    depends_on:
      - nextjs
```

Et une configuration Nginx simplifiée (`nginx-initial.conf`):

```nginx
server {
    listen 80;

    # Proxy vers l'application Next.js
    location / {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;

        # Timeouts plus longs pour les opérations qui durent longtemps
        proxy_read_timeout 60s;
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
```

```
    }

    # Mise en cache des assets statiques
    location /_next/static/ {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;

        # Cache pour les fichiers statiques
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
        access_log off;
    }

    # Mise en cache des images statiques
    location /static/ {
        proxy_pass http://nextjs:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;

        # Cache pour les images et autres médias
        expires 30d;
        add_header Cache-Control "public, max-age=2592000";
        access_log off;
    }
}
```

4. Déploiement de l'application

Transfert des fichiers vers le serveur

À partir de votre machine locale, transférez les fichiers nécessaires:

```
# Création d'un dossier pour regrouper les fichiers à transférer
mkdir deploy-next && cd deploy-next

# Copie des fichiers nécessaires
cp ../Dockerfile ../docker-compose.yml ../docker-compose-initial.yml .
cp ../.env.production ../nginx.conf ../nginx-initial.conf .
cp -r ../prisma .

# Compression des fichiers
tar -czvf deploy-next.tar.gz *

# Transfert vers le serveur
scp deploy-next.tar.gz ubuntu@VOTRE_IP_PUBLIQUE:~/app/
```

Installation sur le serveur

Sur votre serveur OVH:

```
cd ~/app
tar -xzvf deploy-next.tar.gz

# Création des dossiers nécessaires
mkdir -p nginx/conf.d nginx/ssl nginx/logs nginx/certbot

# Copie de la configuration Nginx initiale
cp nginx-initial.conf nginx/conf.d/default.conf

# Démarrage des conteneurs (version initiale sans SSL)
docker-compose -f docker-compose-initial.yml up -d
```

Vérification du déploiement initial

Ouvrez votre navigateur et accédez à `http://VOTRE_IP_PUBLIQUE:80` Votre application Next.js devrait être accessible.

5. Configuration DNS et SSL (après obtention du domaine)

Une fois votre nom de domaine configuré pour pointer vers l'IP du serveur:

```
# Arrêt des conteneurs
docker-compose -f docker-compose-initial.yml down

# Copie de la configuration Nginx complète
cp nginx.conf nginx/conf.d/default.conf

# Préparation pour Let's Encrypt
mkdir -p nginx/certbot

# Démarrage avec la configuration complète
docker-compose up -d
```

6. Scaling et Monitoring

Mise en place du monitoring

Créez un fichier `docker-compose-monitoring.yml` :

```yaml
version: "3.8"

services:
  prometheus:
    image: prom/prometheus
    restart: always
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus:/etc/prometheus
      - prometheus_data:/prometheus
```

```yaml
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"
      - "--web.console.libraries=/usr/share/prometheus/console_libraries"
      - "--web.console.templates=/usr/share/prometheus/consoles"

  grafana:
    image: grafana/grafana
    restart: always
    ports:
      - "3001:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus

  node-exporter:
    image: prom/node-exporter
    restart: always
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - "--path.procfs=/host/proc"
      - "--path.sysfs=/host/sys"
      - "--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)
($$|/)"

volumes:
  prometheus_data:
  grafana_data:
```

7. Maintenance et mises à jour

Mettre à jour l'application

```bash
# Récupération des nouvelles modifications
scp deploy-next.tar.gz ubuntu@VOTRE_IP_PUBLIQUE:~/app/

# Sur le serveur
cd ~/app
tar -xzvf deploy-next.tar.gz

# Rebuild et redémarrage des conteneurs
docker-compose build nextjs
docker-compose up -d
```

Sauvegarder la base de données

Si vous utilisez une base de données externe comme PostgreSQL:

```
# Sauvegarde avec pg_dump
docker run --rm -v $(pwd)/backups:/backups postgres:13 \
  pg_dump "postgres://user:password@host:port/database" > /backups/backup-$(date
+%Y%m%d).sql
```

8. Scalabilité

Options de scaling vertical

1. Augmentez les ressources de votre instance OVH via le portail Public Cloud
2. Redémarrez votre instance pour appliquer les changements

Options de scaling horizontal

1. Créez plusieurs instances OVH
2. Configurez un Load Balancer OVH
3. Modifiez votre configuration pour utiliser un stockage partagé pour les uploads
4. Utilisez une base de données capable de gérer plusieurs connexions

9. Annexes

Optimisation des performances

- Activez la compression Gzip dans Nginx
- Utilisez un CDN pour les assets statiques
- Configurez le cache au niveau du navigateur avec les en-têtes appropriés

Sécurité renforcée

- Configurez un pare-feu avec ufw ou iptables
- Mettez en place un scan de vulnérabilités automatique
- Configurez des sauvegardes automatiques

Automatisation CI/CD

- Configurez un pipeline GitHub Actions
- Automatisez les tests et le déploiement

Ce guide vous a fourni toutes les étapes nécessaires pour déployer une application Next.js sur OVH Public Cloud de manière professionnelle, sécurisée et scalable.